COMP8210
Big Data Technologies

Assignment 1

Semester 2, 2026
Macquarie University, School of Computing

Due: Friday 18 September, at 5pm
Demonstration in Lab: During Week 8 (14 September to 18 September)
Total Mark: 100%
Weighting: 35%

This Assessment Task relates to the following Learning Outcomes:

- ULO1: Demonstrate a high level of technical competency in standard and advanced methods for big data technologies
- ULO2: Describe the current status of and recognize future trends in big data technologies
- ULO5: Communicate clearly and effectively

This assessment task has 2 main Sections. The first Part (**Section A - 30%**) focus on Document Databases (MongoDB) and the second part (**Section B - 70%**) will focus on Graph Databases (Neo4J).

Section A- Document Databases (30%)

Part 1 (Sec A) . MongoDB on Twitter Dataset (30%)

Background.
Social media platforms such as X (Twitter) represent a rich source of information for understanding human behaviour, communication patterns, and coordinated online activity. The rapid, high-volume nature of tweets provides opportunities to study emerging topics, track the diffusion of ideas, detect misinformation, and identify suspicious or coordinated behaviours across groups of users. Insights derived from this type of data include:

- **Content trends** – measuring the rise and fall of hashtags, keywords, and domains to detect emerging narratives.
- **User behaviour** – identifying prolific users, repeated posting of near-duplicate content, or unusual temporal activity patterns.
- **Coordination signals** – uncovering groups of users who share attributes (e.g., location, URLs, hashtags) or who post in tightly synchronized bursts.
- **Network structures** – exploring how mentions, retweets, or shared links create implicit networks that may reveal influence or manipulation.

Dataset.
To support such analyses, the Twitter dataset has been curated into a **document-oriented schema** suitable for MongoDB. Each tweet is represented as a JSON document, with important fields extracted and normalised:

- **Tweet-level fields:**
  - id_str (string, unique tweet identifier)
  - created_at (ISODate timestamp)
  - text (raw text), clean_text (URLs/mentions removed)
  - hashtags (array of lowercased tags)
  - mentions (array of screen_names)
  - urls (array of objects: {expanded_url, domain})
  - hour_of_day, weekday, has_link (derived fields for temporal and behavioural analysis)

© https://data-science-group.github.io/

- **User-level fields (nested or separated into users collection):**
  - user_id (unique identifier)
  - screen_name, name, location
  - url_domain (domain of the profile URL, if present)

This schema design allows efficient use of MongoDB's aggregation framework, supporting both **temporal queries** (via indexed timestamps) and **content-based queries** (via arrays such as hashtags and mentions). By separating users into their own collection and maintaining relational integrity through user_id, we enable more advanced join-style queries that capture coordination and community-level patterns.

Small Sample: A 50-tweet JSON is provided to test your pipeline and queries before running on the 10k dataset.

Tasks. Use MongoDB (≥5.0) and the provided Twitter dataset (10k tweets; a 50-tweet sample is also provided) to build a small curation pipeline and answer challenging analytical queries.

Part A (10%) — Data Curation Pipeline (Python → Mongo)

Write a **Python** ingestion/cleaning script that reads the raw JSON and writes to Mongo collections (e.g., tweets_raw → tweets_clean, users). Use the technologies, such as MapReduce (Refer to Course Prerequisites such as the Big Data Unit), to help you scale the processing pipeline.

a) Parse fields and normalise schema:
   - created_at → ISODate (preserve original timezone), id_str → string PK; user.id → user_id.
   - Extract arrays: hashtags (lowercased), mentions (screen_names), urls (expanded + domain).
   - Add derived fields: hour_of_day (0–23), weekday, has_link (bool), clean_text (strip URLs/mentions), lang_guess (simple heuristic allowed).

b) **Deduplicate** by id_str; keep the latest by created_at.

c) Split and upsert a users collection (user_id, screen_name, name, location, url_domain).

d) Create indexes:
   - tweets_clean: {created_at:1}, {hashtags:1, created_at:1}, {user_id:1, created_at:1}, {urls.domain:1, created_at:1}.
   - users: {user_id:1} unique, {screen_name:1}.

Deliverables: script, example load command, and a brief note (≤150 words) justifying index choices.

Part B (10%) — Content & Temporal Analytics (Aggregation Pipelines)

Run queries on tweets_clean only; return **concise tables**.

a) **Emerging Hashtags (WoW growth).** For the last two full weeks in the data, compute each hashtag's weekly counts and return the **top 10** by growth rate (week2_count / week1_count), with week1_count ≥ 20 and week2_count ≥ 40. Columns: hashtag, week1_count, week2_count, growth_rate. (Use $unwind, $dateTrunc, $group, $lookup/$setWindowFields.)

b) **Coordinated Link Bursts.** Detect domains that see **≥3 distinct users** posting within any **60-second** window. Return: domain, window_start, unique_users, tweet_ids_sample (up to 5). (Use $match on has_link, $setWindowFields with range on created_at, then group by domain + window bucket.)

c) **High-volume Near-Duplicates per User.** For each user/day, flag users who post **≥5 tweets** whose clean_text is exactly identical after URL/mention stripping. Return: user_id, screen_name, day, duplicate_text, dup_count. (Use $dateTrunc, $group on {user_id, day, clean_text} and filter.)

Part C (10%) — Advanced Integrity & Coordination Detection

a) **Sock-puppet Signals (Intersection-based).** Find **pairs of users** who (a) share a **location OR url_domain** in users, and (b) within any **24-hour** period share **≥3 common hashtags**. Use a self-join ($lookup users → tweets, and tweets → tweets) and set operations ($setIntersection). Return: user_a, user_b, shared_attr (location/url_domain), day, shared_hashtags, support_count.

b) **Cross-account Promotion Chains.** Build **account→account** promotion edges when **User A** links to a domain that **User B** also posts **within 2 minutes** and includes an **@mention of A**. Use $lookup with time-bounded join on created_at and filter by mentions. Return top 10 edges by frequency with: src_user, dst_user, events, median_latency_sec. *(Hint: compute latency with $dateDiff; summarise via $group and $percentile or $avg.)*

Notes (Parts B–C):

- All answers must be **single aggregation pipelines** (you may create temporary collections/materialized views if justified).
- Use your indexes from Part A; include a brief **explain() screenshot** for one query showing an index scan/seek.
- Treat text case-insensitively where appropriate (set **collation** or normalise in curation).

Section B – Graph Databases (70%)

Background.
With scams and fraud costing Australians over $3B in 2022, increasingly fraud actors are becoming more sophisticated and difficult to detect with traditional techniques. Graph data science powers a new generation of fraud solutions, designed to leverage connections in data to detect behavioural patterns and trace actions through complex networks of operators.

Dataset.

We have supplied a number of data files on iLearn representing a set of accounts and money transfers across a fictitious set of individuals and retailers/sellers. There are a series of CSV formatted files as follows:

clients.csv - contains the account details for the individual account holders in the system. This includes synthetically generated id, name, contact details and tax file number.

stores.csv - id and names off the sellers of stores that users are purchasing from.

purchase.csv - amount and time of a purchase made by an individual from a seller

xfer.csv - amount and time of a money transfer made between two users in the system.

You will use this data to construct a graph of interconnections between users and stores in the system, you will also use a graph to analyse the account details of users in the context of fraud.

Part 1. Initial Graph Data Model (30%)

Task. Create an enriched base graph data model for the payments system by importing the CSV data into Neo4j. Instead of a straightforward import, you are required to **design, justify, and validate** your modeling choices with attention to scalability, query efficiency, and extensibility.

Requirements:

1. **Modeling Extensions**
    - In addition to the structure shown in Figure 1, introduce at least **two additional relationship types** derived from the dataset (e.g., temporal ordering of transactions, client–client similarity via shared attributes, or inferred "household"/"business cluster" groupings).
    - Both Purchase and Transfer must be modeled as subtypes of Transaction, but also ensure your design supports potential future transaction types (e.g., Refund, Loan, CryptoTransfer).
2. **Temporal Representation**

- Convert the timeOffset field into an **absolute temporal property** (date and time), but also store at least one **derived temporal feature** (e.g., hour of day, weekday/weekend, business hours flag) to enable more complex fraud pattern analysis later.
- Justify which temporal properties are indexed and why.

3. **Constraints and Indexing Strategy**
   - Create appropriate **unique constraints** and **indexes**, but also explain your design trade-offs: which fields you indexed and why (e.g., frequent query filters, join points, or GDS projections).
   - Demonstrate how your indexing decisions affect query planning by comparing two query plans (with/without index).

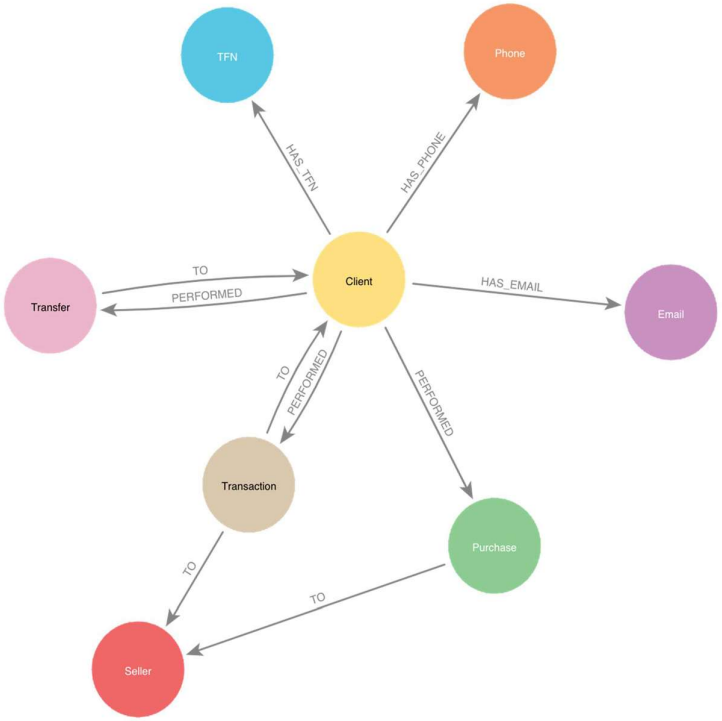


Figure 1. Graph Data Model for payments system.

4. **Data Integrity and Provenance**
   - Add a :Source node type to represent the provenance of each imported dataset (e.g., purchase.csv, xfer.csv). Connect transactions to these source nodes. This prepares the graph for handling multi-source datasets in later stages.
5. **Documentation of Design Choices**
   - Alongside your Cypher import scripts, submit a short **data modeling rationale (max 500 words)** describing:
     - Why you introduced the new relationships or properties.
     - How your temporal modeling supports fraud detection use cases.
     - Which constraints/indexes you chose and the trade-offs involved.

Part 2. Initial Queries (20%)

Write Cypher statements, in Neo4j, for each of the following problems:

- **Problem 1 (5%)**
  Which client(s) spent the most on **Purchase** transactions between **10:00 and 14:00 (AEST) on 12 May 2024**, using the absolute timestamp converted from timeOffset.

Requirements:

- ✓ Query must be **parameterised** for start/end time.
- ✓ **Exclude** zero/negative amounts.
- ✓ **Handle ties** (return all clients with the top total; include a dense rank).
- ✓ Return columns: name, total_in_window, txn_count, first_txn_time, last_txn_time, top_store, top_store_spend, rank.

- **Problem 2 (5%)**
  Identify the **top 5 clients** with the **largest negative cashflow**, where total **outgoing (purchases + transfers sent)** exceeds total **incoming (transfers received)**.

Requirements:

- ✓ Compute balance = incoming – outgoing.
- ✓ Include max_outgoing = the client's **highest single outgoing transaction**.
- ✓ Show total_outgoing, total_incoming, and txn_count (all transactions).
- ✓ If multiple clients tie at rank 5, return them all.
- ✓ Return columns: name, balance, max_outgoing, total_outgoing, total_incoming, txn_count.

- **Problem 3 (5%)**
  We suspect the Seller **'Woods'** receives pass-through funds from scam activity. Identify **"pass-on" clients** who receive transfers and then spend part of that money at Woods.

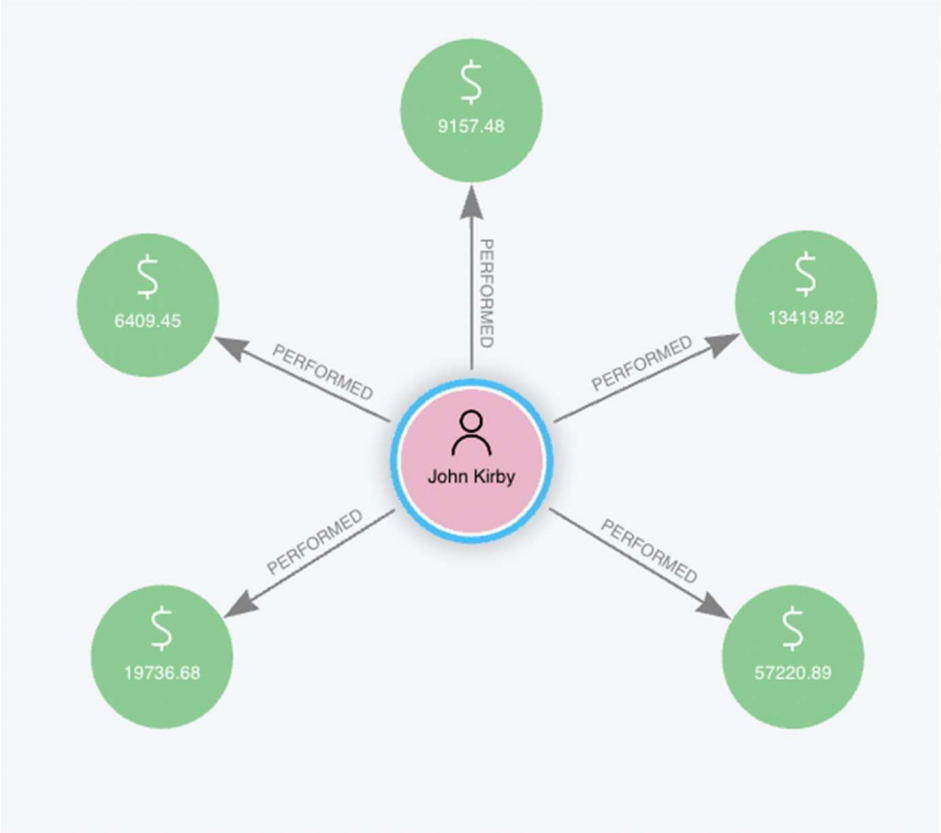
Requirements:

- ✓ Parameterise seller and lookback window: $sellerName (default "Woods"), $lookbackHours (e.g., 48).
- ✓ For each client, consider only **transfers received strictly before** each Woods purchase and **within** the $lookbackHours window.
- ✓ Compute:
  - attributed_xfer = sum of qualifying transfers (lookback-window).
  - total_purchase = sum of purchases at $sellerName.
  - percentage = (total_purchase / attributed_xfer) × 100.
- ✓ Exclude zero/negative amounts and **self-funding** (ignore transfers from the same client).
- ✓ Return only clients with percentage ≥ 5 and at least **2 purchases** at $sellerName.
- ✓ Return columns: name, percentage, attributed_xfer, total_purchase, first_purchase_time, n_senders, top_sender, top_sender_amount.

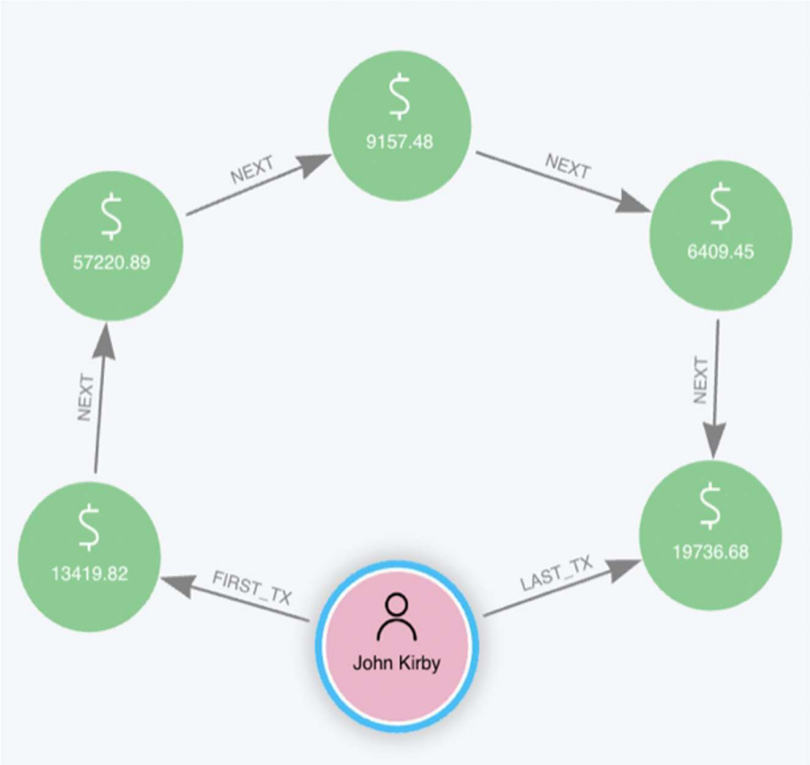
- **Problem 4 (5%):**

  We think that there are repeated patterns of transactions that might serve as a signature for fraudulent behaviour. To support future use cases like this we would like to add a new layer of relationships to the data set that creates an ordering of transactions. Currently, Clients are related to transactions in the following way :

© https://data-science-group.github.io/



Write GQL/Cypher statements to create new relationships that connect ordered transaction as follows:



© https://data-science-group.github.io/

Part 3. Graph Data Science (20%)

Now we will use Neo4j Graph Data Science (GDS) to analyse advanced features of the client–transaction graph and extend our fraud detection pipeline.

Problem 1 (6%)

From the original data model, clients are linked via shared details (email, phone, tax file number, etc.). Stolen or duplicated details often produce unusually dense clusters of accounts.

a) Use Neo4j GDS to create a **heterogeneous graph projection** including both Clients and their shared identifiers. Apply at least **two different community detection algorithms** (e.g., Louvain and Connected Components) and compare their output.
b) Identify the larger groups (≥5 members), assign a persistent fraud_group_id, and write back both the algorithm used and the group size to the database for each Client.
c) For the largest group, create a **visualisation** in Neo4j Browser or Bloom showing all Clients and shared identifiers, styled by group size and detection method.

Problem 2 (8%)

Detecting groups is valuable, but the more difficult task is tracing **how funds flow into and out of these groups**.

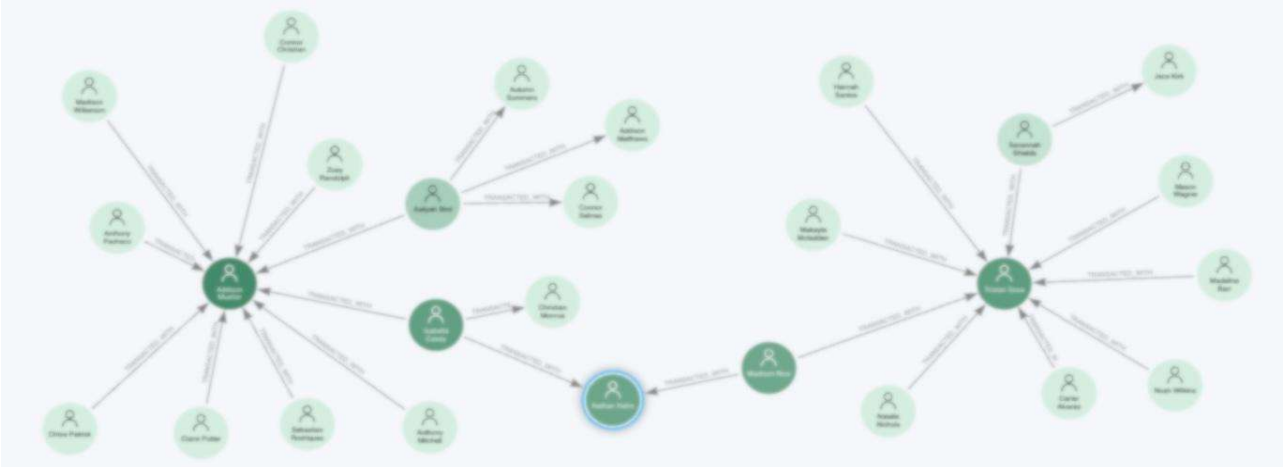
a) Write a Cypher query that isolates **cross-boundary transactions**, i.e., transfers where the sender is inside a fraud group but the recipient is outside, and return the top 10 recipients ranked by total incoming amount.
b) Build a **subgraph projection** of these boundary accounts (fraud group members + their external connections). Use a clustering algorithm (e.g., Label Propagation or k-core) to detect tightly connected subsets that may represent organised funnels.
c) On the largest detected subset, run at least **two centrality measures** (e.g., PageRank and Betweenness) to identify which accounts appear to control or route the majority of funds. Compare the results and justify which measure gives the most reliable "key suspect" in this scenario.
d) Create a Bloom visualisation of the final result with **value-based styling** to highlight the most central accounts, and annotate the visualisation with group IDs and transaction volumes for interpretability.

Problem 3 (6%)

Fraud networks often adapt to detection by hiding activity across multiple layers. To push deeper:

a) Construct a **temporal transaction graph** projection where relationships encode both value and time (e.g., [:TRANSFER {amount, timestamp}]).
b) Apply a **link prediction or similarity algorithm** (e.g., Node Similarity, Jaccard, or GDS Link Prediction pipelines) to identify pairs of Clients **likely collaborating in fraud** but not directly transacting yet.
c) Select the top 5 predicted links with highest fraud-likelihood score and explain why these may indicate hidden connections.
d) Create a **timeline visualisation** (e.g., using Bloom with time slider or exported snapshots) that shows how suspicious connections evolve over time and highlight where predicted links suggest emerging fraud clusters.

© https://data-science-group.github.io/



Evaluation and Marking

- This is an individual assignment worth 35%.
- All assignments will be submitted using iLearn. The results of all assignments will be available via iLearn.
- You will need to create a video (max 10 minutes) and upload it on YouTube. Then, share the YouTube link in your assignment submission.
- The submission will be a zip file including the source code for Part 1 and 2, the YouTube Link, and the queries for part 3. You do not need to include the Graph database in the submission file.
- Students will demonstrate their assignments **during Week 8 (14 September to 18 September)** in person (in Practical and SGTA sessions).
- No late submissions will be accepted, unless a Special Consideration is Submitted before the assessment submission deadline, and Granted. Your assignment will be evaluated by the tutor independently.
- **If you have any questions related to this assignment, please submit them on the Discussion Forum on iLearn.**

© https://data-science-group.github.io/